

Additional Methodological Details

This document details the expectation-maximization (EM) algorithm and the gradient boosting algorithm by providing line-by-line explanations. These algorithms may use two predefined functions (detailed in Section 3):

- (1) **CraftSystemPrompt**: Given a target value x and the description of the social context c , this function leverages an LLM (e.g., GPT) to craft a system prompt s that reflects this observed social activity or attribute x (see Section 3.1 for details).
- (2) **GenerateSamples**: Given the crafted system prompt s and the social context c , this function generate n independent simulated samples (see Section 3.2 for details).

1 Line-by-Line Explanations for the EM Algorithm

Initialization: We initialize the K system prompts by sampling K distinct data points $\{x_k\}_{k=1}^K$ from $\mathcal{D}_{\text{obs}}$ and crafting prompts to reflect the sampled attributes (calling **CraftSystemPrompt**(x_k, c), see Appendix 3.1 for more details).

Main loop: The algorithm proceeds iteratively until convergence, alternating between the E-step and M-step. The algorithm is considered converged when the system prompts $\mathcal{S}$ and weights $\mathbf{w}$ remains unchanged between consecutive iterations.

E-step (Expectation): Assigning observed data to prompts. For each observed data point $x_i \in \mathcal{D}_{\text{obs}}$:

- For each system prompt s_k , generate samples with an LLM, i.e., $\{x_j\}_{j=1}^n = \text{GenerateSamples}(s_k, c, n = 10)$. Then compute the Wasserstein distance between x_i and the generated samples as $d_k = \frac{1}{n} \sum_{j=1}^n d(x_i, x_j)$.
- Convert these distances $\{d_k\}_{k=1}^K$ into assignment probabilities using a softmax-like transformation: $\{p_k \propto w_k \cdot \exp(-d_k)\}$.
- Sample a prompt assignment Z_i for x_i from the categorical distribution defined by $\{p_k\}_{k=1}^K$.

M-step: Updating system prompts and weights. For each system prompt s_k :

- Aggregate the data points assigned to it, $\{x_i \mid Z_i = k\}$.
- Sample a desired attribute x from the assigned data points.
- Craft a new system prompt s_k towards generating behaviors similar to x (calling **CraftSystemPrompt**(x, s), more details are in Appendix 3.1).

In weight optimization, define the loss function $\mathcal{L}(\mathcal{S}, \mathbf{w})$ as the Wasserstein distance between the observed distribution $\mathcal{D}_{\text{obs}}$ and the model-simulated distribution $\mathcal{D}_{\text{sim}}(\mathcal{S}, \mathbf{w})$. Use a constrained optimizer to solve for the weights $\mathbf{w}$ that minimize this loss. More details for the loss function and the optimization process is in Appendix 4.

Return: The final learned system prompt set $\mathcal{S}$ and the optimized weights $\mathbf{w}$ together define a prompt mixture that closely approximates the target human distribution $\mathcal{D}_{\text{obs}}$.

2 Line-by-Line Explanation of the Gradient Boosting Algorithm

Initialization: A default system prompt s_0 is initialized as the default system prompt “You are a helpful assistant,” and its weight w_0 is set to 1. This serves as the starting point of the boosting process.

Main loop: For each iteration from $k = 1$ to K :

- **Generate samples** $\mathcal{D}_{\text{sim}}$ using the current mixture of system prompts $s_0, \dots, s_{k-1}$ (calling **GenerateSamples**($s_k, c, n = 10$) for each prompt). The samples are combined according to their corresponding weights to form an estimation of the simulated distribution $\mathcal{D}_{\text{obs}}$, which represents the model’s current approximation.
- **Compute residual** r by comparing the generated distribution $\mathcal{D}_{\text{sim}}$ with the observed distribution $\mathcal{D}_{\text{obs}}$. The residual r is a new distribution across the attribute space. Practically, we compute it as $p(x) = \max(p_{\text{obs}}(x) - p_{\text{sim}}(x), 0)$ and normalize it to integral to 1. This residual reflects aspects of the target behavior not yet captured by the current prompt ensemble.
- **Sample desired attribute** x from the residual r . This selects an attribute the current model mixture underrepresents.
- **Craft new prompt** s_k that steers the LLM towards producing attributes similar to x . (calling **CraftSystemPrompt**(x, c), see Appendix 3.1 for more details).
- **Add** s_k to the prompt set $\mathcal{S}$.
- **Define optimization objective**: Use Wasserstein distance between $\mathcal{D}_{\text{obs}}$ and the newly simulated $\mathcal{D}_{\text{sim}}(\mathcal{S}, \mathbf{w})$ as the loss function to optimize w_k . More details for the loss function is in Appendix 4.
- **Optimize weight** w_k to minimize the loss $\mathcal{L}$, using Sequential Least Squares Programming (SLSQP). More details for the optimization process is in Appendix 4.
- **Add** w_k to the list of weights.

Return: The final set of system prompts $\mathcal{S}$ and their optimized weights $\mathbf{w}$ form the distributional alignment strategy learned by the gradient boosting algorithm.

3 Details of Predefined Functions

Below, we define two utility functions: **CraftSystemPrompt** and **GenerateSamples**. For these functions, we use gpt-4o-2024-05-13 for the Big Five and MobLab datasets, and gpt-4o-2024-08-06 for the World Values Survey. All API parameters are left at their default values, except for n , which is adjusted according to the requirements of each function call.

3.1 CraftSystemPrompt(x, c)

This function crafts a system prompt s that guides an LLM to generate outputs reflecting the desired attribute x under the social context c , i.e., $x \sim p(x \mid s, c; \mathcal{M})$. It consists of two stages: the initial crafting and iterative improving. In the crafting stage, a candidate system prompt is generated. If it does not successfully reflect the

desired attribute in the `GenerateSamples` function, the improving stage is triggered to further refine the prompt in an iterative way.

Crafting: This stage crafts the initial candidate system prompt with the following prompt template. It calls the LLM API, with the argument `n` set to 1 and all other parameters kept at their default values. Below is the prompt template:

```
# Goal
Assuming a chatbot is [a description of the setting, e.g.,
playing games]. Your goal is to craft a system prompt for this
chatbot, so that with the crafted system prompt, the chatbot
behaves in a certain way.

# Game Instruction
[a description of the social context c, e.g., game instruc-
tion or survey questions]

# Desired Behavior
For this particular question/game, please generate a system
prompt for the chatbot. With the generated system prompt
and the above game instructions provided, the chatbot should
[desired attribute x].

# Crafting Requirements
* The ultimate goal is to prompt the chatbot to behave towards
the desired behavior under the given scenario, rather than be-
ing a precoded helpful assistant chatbot.
* Craft the system prompt based on understanding the meaning
of the desired behavior under the given game scenario.
* Do not include information already included in the game in-
structions.
* Craft a generalizable system prompt and avoid including any
information specific to this particular game or directly implying
the desired behavior.

# Output Format
Directly output the crafted system prompt starting with "You
are ...".
```

Improving: The following template is used in the improving stage, where messages from the crafting stage are retained within the same chat completion session. This stage calls the LLM API with `n` set to 1 and all other parameters left at their default values.

```
Using your crafted system prompt, a chatbot outputs mostly
[mode of GenerateSamples(s, n = 10), where s is the sys-
tem prompt obtained from last craft/iteration] instead of
[desired attribute x]. Do you have any idea how to improve
the system prompt?

# Crafting Requirements
[the same as in the crafting prompt]

# Output Format
Directly output the crafted system prompt starting with "You
are ...".
```

3.2 GenerateSamples(s, c, n)

This function uses an LLM to generate n samples $\{x_i\}_{i=1}^n$ given a crafted system prompt s and the social context c , i.e., $x_i \sim p(x | s, c; \mathcal{M})$. The generated samples can be used to estimate the simulation distribution. For some settings (e.g., Big Five personality test and World Values Survey), the model outputs $\mathcal{M}(s, c)$ will need to be further aggregated into an attribute value (see the “Generating simulations” paragraph in Sec. ?? for details).

This function calls the LLM API with `n` set to 10 (i.e., generating 10 independent samples), and all other parameters left at their default values. The system prompt is set as s , while the user prompt is set to c .

4 Loss Function and System Prompt Weight Optimization

Our optimization objective is defined as the Wasserstein distance between the observed distribution $\mathcal{D}_{\text{obs}}$ and the model-simulated distribution $\mathcal{D}_{\text{sim}}(\mathcal{S}, \mathbf{w})$, where $\mathcal{S} = \{s_k\}_{k=1}^K$ is the collection of system prompts, and $\mathbf{w} = \{w_k\}_{k=1}^K$ are their corresponding weights. The generated distribution $\mathcal{D}_{\text{sim}}(\mathcal{S}, \mathbf{w})$ is obtained by sampling $n = 10$ responses from the LLM for each prompt s_k , resulting in a weighted distribution aggregated according to the normalized weights $\mathbf{w}$, i.e., $p(x) = \sum w_k \cdot p(x | s_k, c; \mathcal{M})$.

To optimize the weights $\mathbf{w}$, we implement a self-contained optimization procedure using Sequential Least Squares Programming (SLSQP), which is well-suited for constrained optimization problems. Specifically, we solve:

$$\min W(\mathcal{D}_{\text{obs}}, \mathcal{D}_{\text{sim}}(\mathcal{S}, \mathbf{w})) \quad \text{s.t.} \quad w_k \geq 0, \sum_{k=1}^K w_k = 1$$

The optimizer is initialized with normalized random weights, with each dimension sampled from a uniform distribution, and we set the tolerance `tol` to 10^{-6} .